Case Study Report: The Self-Pruning Neural Network

Abstract

This project presents a self-pruning neural network for CIFAR-10 image classification. The model is designed so that it can learn to remove unimportant connections during training itself, instead of pruning only after training. This is achieved by adding a learnable gate to every weight and using an L1 sparsity penalty to encourage many gates to become close to zero. The final model is evaluated using test accuracy and sparsity level to study the trade-off between compression and performance.

Introduction

In practical deep learning systems, large networks often face limits in memory, speed, and computation. Pruning is a common solution that removes unnecessary weights or neurons to make the model smaller and faster. Traditional pruning methods are usually applied after training, but this case study explores a more advanced idea: the network should learn to prune itself while it is learning.

The key concept is that every weight has a corresponding gate value between 0 and 1. A gate near 1 keeps the connection active, while a gate near 0 suppresses it. This allows the network to automatically decide which connections are important and which are not.

Problem Statement

The task was to build a neural network for CIFAR-10 that includes:

- a custom PrunableLinear layer,
- a learnable gate_scores tensor for each weight,
- a sparsity regularization loss,
- a training loop that updates weights and gates together,
- and an evaluation process that measures both test accuracy and sparsity.

The model must compare at least three different values of λ to show how sparsity and accuracy change.

Methodology

Prunable Linear Layer

A custom linear layer was implemented from scratch instead of using the built-in fully connected layer. Each layer contains:

- weight: the normal weight matrix,
- bias: the bias vector,
- gate_scores: a second trainable matrix with the same shape as the weights.

During the forward pass:

1. gate_scores are passed through the sigmoid function.
2. This produces gate values between 0 and 1.
3. The gate values are multiplied element-wise with the weights.
4. The resulting pruned weights are used for the linear transformation.

This makes the layer differentiable, so gradients flow through both the weights and the gate scores.

Sparsity Loss

To encourage pruning, the model uses an extra penalty term:

$$\text{Total Loss} = \text{Classification Loss} + \lambda \times \text{Sparsity Loss}$$

The sparsity loss is the sum of all gate values. Since sigmoid outputs are always positive, this behaves like an L1 penalty. L1 regularization encourages many values to become very small, and in this case it pushes unnecessary gates toward zero.

Training Procedure

The model was trained on CIFAR-10 using the Adam optimizer. For each mini-batch:

- the images were passed through the model,
- classification loss was computed using cross-entropy,
- sparsity loss was computed from the gates,
- the total loss was formed,
- backpropagation updated both weights and gate scores.

The model was trained with three different values of λ:

- 0.0,
- 0.001,

- 0.01.

This was done to observe the trade-off between accuracy and sparsity.

Results

Training Behavior

From the output, the model with $\lambda = 0.0$ trained well on the training data. Training accuracy increased steadily from about 41.96% to 89.20% over 20 epochs. However, test accuracy reached only 52.77%, which indicates overfitting. The sparsity level remained 0.00%, meaning no weights were actually pruned in that case.

Result Table

Lambda	**Test Accuracy**	**Sparsity Level**
0.0	52.77%	0.00%
0.001	Not shown in output	Not shown in output
0.01	Not shown in output	Not shown in output

The visible output only clearly showed the $\lambda = 0.0$ case. The remaining results should be filled in after running the other two experiments.

Discussion

The results show that the model can learn classification reasonably well, but pruning does not happen when $\lambda = 0.0$. This is expected because there is no sparsity pressure on the gates. The test accuracy being much lower than training accuracy suggests the model is overfitting.

The sparsity mechanism is working in concept, but the current setup needs tuning to produce stronger pruning. A higher λ should create more gates near zero, but too much regularization may reduce accuracy. This is the main trade-off in the project.

The gate distribution plot helps verify pruning behavior. A successful self-pruning model should show:

- a strong spike near 0,
- and another group of values away from 0.

If the plot does not show this shape, pruning is not yet strong enough.

Conclusion

This case study successfully demonstrates the structure of a self-pruning neural network. The custom PrunableLinear layer allows every connection to learn its own importance through gate scores. The L1 sparsity loss encourages unnecessary weights to be suppressed. The training results show that the model can learn the classification task, but the current configuration still needs tuning to achieve better test accuracy and meaningful sparsity.

The most important conclusion is that self-pruning is possible during training itself, and the balance between accuracy and sparsity is controlled by λ. A well-tuned model should keep accuracy reasonably high while also pruning a significant percentage of weights.

Future Work

To improve the system further, the following changes can be explored:

- use a stronger CNN architecture,
- train for more epochs,
- tune λ more carefully,
- apply better data augmentation,
- initialize gate scores to encourage easier pruning,
- use learning-rate scheduling.

Final Summary

This project builds a neural network that learns not only to classify images but also to remove weak connections by itself. It achieves this using gated weights and L1 regularization. The approach is useful for compact and efficient neural networks where memory and computation are limited.